GUARD-RAG: Persistent Answer-Component Coverage Tracking for Completeness-Aware Retrieval-Augmented Generation

GUARD: Guided Understanding and Adaptive Retrieval with Evidence Assurance

Abstract

Retrieval-Augmented Generation (RAG) systems can retrieve relevant passages and still produce incomplete answers, especially for questions with multiple answer components or exhaustive entity lists. A single relevance score does not reveal which required components remain unsupported, and a single context-level sufficiency judgment does not say which ones are missing.

This proposal introduces **GUARD-RAG**, a retrieval-control framework that represents a query's requirements as explicit answer components and maintains an evidence-support state (missing, partial, supported, unresolved) for each component across retrieval iterations. The controller chooses its next retrieval action from these states, which is the sense in which the system is adaptive, and stops when predefined conditions are met. If evidence or budget runs out, it returns a partial answer that names the unresolved requirements instead of implying completeness.

We will compare GUARD-RAG with fixed and hybrid RAG, Adaptive-RAG, a corrective baseline, IRCoT, and three controlled variants that share its retriever, generator, and budget: decomposition-only, sufficiency-only, and a compute-matched baseline with no coverage state. An oracle-requirement experiment isolates requirement-extraction errors. The primary metric is item-level F1 on MuSiQue, QAMPARI, and a new corpus-bounded annotated set, the GUARD-RAG Completeness Set. The central hypothesis is that persistent component-level coverage tracking improves answer completeness and stopping decisions beyond decomposition, context-level sufficiency, and simply retrieving more. Novelty and effectiveness will be established empirically, and negative results will be reported.

1. Introduction

RAG grounds language-model responses in retrieved documents, but retrieving relevant information does not mean retrieving everything needed. Three properties must be kept apart:

- **Relevance:** is the retrieved information related to the question?
- **Evidence sufficiency:** does the evidence adequately support a particular claim?
- **Answer completeness:** have all required components been addressed, within the defined scope of the question and corpus?

They are related but not interchangeable. A passage may be relevant yet support only one part of a multi-part question, and evidence may support an individual claim without showing that every required claim is covered.

Prior work supplies important building blocks. Adaptive-RAG selects retrieval strategies by question complexity; Self-RAG adds retrieval decisions and self-reflection; Corrective RAG (CRAG) evaluates retrieval quality and triggers corrective actions; FLARE retrieves actively during generation; IRCoT interleaves retrieval with chain-of-thought reasoning; and Sufficient Context studies context adequacy and selective answering. QAMPARI and QUEST are benchmarks for questions with many answers or implicit set constraints.

GUARD-RAG asks a narrower question: **can persistent tracking of evidence coverage for each required answer component improve retrieval control beyond question decomposition, context-level sufficiency judgments, or conventional iterative retrieval?** We do not assume that prior methods lack every form of component tracking. A systematic review and controlled experiments will decide whether the proposed representation and controller are a distinguishable contribution.

2. Problem Statement

Consider a query requiring five distinct answer items. A conventional pipeline may retrieve strong evidence for three and generate a fluent answer without noticing the other two are missing. A relevance score does not show that five requirements were covered, and a context-level sufficiency judgment may not identify which individual requirements remain unsupported.

GUARD-RAG represents query requirements as $R(q) = \{r1, r2, ..., rn\}$. For each requirement the system maintains one of four states:

- **Missing:** no adequate supporting evidence has been identified, and no targeted retrieval has yet been attempted for it.
- **Partial:** some relevant evidence exists but does not yet meet the support criteria.
- **Supported:** the available evidence meets predefined support criteria.
- **Unresolved:** targeted retrieval has been attempted and the requirement still cannot be established, because the budget is exhausted, the corpus lacks adequate evidence, or the evidence is ambiguous or conflicting.

The controller uses these states to pick the next retrieval action, decide whether more retrieval is worthwhile, and decide whether the final answer is complete or explicitly partial.

Research problem. Under matched retrieval, generation, and computational budgets, does persistent answer-component coverage tracking improve answer completeness and stopping decisions compared with decomposition-only, sufficiency-only, compute-matched, and established iterative or adaptive approaches?

3. Research Gap and Positioning

The intended contribution is not another adaptive RAG pipeline but a test of whether a persistent component-level evidence state is a useful control signal. Four distinctions guide the study:

1. **Decomposition vs. persistent coverage.** Decomposition creates sub-questions. GUARD-RAG also keeps their evidence states across iterations and uses state changes to steer later actions.
2. **Context vs. component sufficiency.** A context-level judgment asks whether the context is adequate overall. GUARD-RAG asks whether each requirement is adequately supported.
3. **Iteration vs. coverage-directed retrieval.** Iteration alone does not ensure the next retrieval targets what remains unsupported.
4. **Stopping vs. completeness-aware stopping.** The stopping rule considers remaining unsupported requirements, evidence quality, diminishing returns, and budget.

These are proposed distinctions, not proof of novelty. The literature review must check methods that already use sub-question tracking, claim-level verification, answer-coverage estimation, iterative retrieval, and stopping policies.

3.1. Method comparison

Method	Established focus	Proposed distinction of GUARD-RAG
Adaptive-RAG	Strategy chosen by question complexity	Persistent evidence state per answer requirement
Self-RAG	Retrieval decisions and self-reflection	Explicit coverage-driven control over unresolved components
Corrective RAG	Retrieval-quality evaluation and corrective actions	Component-specific retrieval priorities and coverage-based stopping
FLARE	Retrieval triggered by low-confidence generation	Stopping and retrieval driven by requirement coverage, not token confidence
IRCoT	Retrieval interleaved with reasoning steps	Persistent per-component state and explicit complete-vs-partial reporting
Sufficient Context	Context adequacy and selective generation	Tracks which individual requirements remain unsupported
GraphRAG / Hyper-RAG	Graph or hypergraph knowledge organisation	No graph construction; studies retrieval-control state

3.2. Evaluation resources

QAMPARI (open-domain questions with many answers) and QUEST (entity-seeking queries with implicit set operations) are evaluation resources, not algorithmic comparators. We use them to test whether a retrieval controller improves answer-item recall and precision.

Adaptive-RAG, Self-RAG, CRAG, IRCoT, and Sufficient Context are essential comparators because their capabilities overlap with parts of our approach. GraphRAG and Hyper-RAG are architectural references and are not fair primary baselines unless their indexing pipelines are adapted to the same setting.

4. Research Questions

- **RQ1.** Does GUARD-RAG improve answer-item F1 and recall over fixed top-k and hybrid RAG?
- **RQ2.** Does persistent component-level tracking reduce premature stopping and unsupported answer items?
- **RQ3.** Does it provide measurable benefit beyond decomposition alone, context-level sufficiency alone, and a compute-matched baseline without coverage state?
- **RQ4.** What trade-offs arise in retrieval calls, tokens, latency, and item precision?
- **RQ5.** Does the benefit vary across multi-hop, multi-part, list, and insufficient-evidence questions?

5. Hypotheses

- **H1:** GUARD-RAG improves item-level F1 and recall over fixed top-k RAG on multi-component tasks.
- **H2:** GUARD-RAG reduces premature stopping and unsupported answer items relative to matched baselines (premature stopping is measured on the Completeness Set only).
- **H3:** The full method outperforms the decomposition-only, sufficiency-only, and compute-matched baselines on the primary metric.
- **H4:** The benefit is largest on questions requiring multiple separately supported components.
- **H5:** Under a predefined resource budget, gains are not explained by extra retrieval alone: at equal retrieval calls and tokens, GUARD-RAG still outperforms the compute-matched baseline.

Pre-specification. The primary metric is **item-level F1**; recall is the main secondary metric, and precision changes are always reported alongside it. Before test evaluation, using development and pilot data only, we will fix the minimum practically meaningful effect size, the confidence-interval procedure, and the multiple-comparison correction. Positive and negative findings will both be reported.

6. Proposed Methodology

6.1. System overview

Four modules:

1. **Requirement extraction:** identify the components a query requires, including sub-questions and list-item requirements.
2. **Evidence retrieval and verification:** retrieve passages and judge whether each supports a specific requirement.
3. **Persistent coverage controller:** maintain requirement states and choose the next retrieval action from unresolved coverage.
4. **Budget-aware generation:** produce a supported answer, or name missing components when evidence or budget is insufficient.

The core method uses a conventional hybrid retriever. Graph or hypergraph indexing is deliberately excluded so the study isolates the controller. The controller's action space is small and interpretable: targeted re-query for a specific component, lexical versus dense retrieval, and query reformulation. "Adaptive" in this proposal means that the next action is selected from current coverage states.

6.2. Requirement representation

For a query q the extractor outputs R_pred(q) = {r1, ..., rn}. Each requirement should be as atomic and independently assessable as practical, and may carry an expected answer type and constraints. The system does not assume every question decomposes into a perfectly known list: extraction may omit requirements, invent unnecessary ones, or split one into several. These errors are measured separately.

6.3. Persistent evidence state

Each requirement keeps: an identifier and description; its current support state; supporting passage identifiers and evidence-to-requirement links; the verification decision; retrieval attempts and newly found evidence; and the reason it remains unresolved, if so. The state persists across iterations. A component is never marked supported merely because a passage is semantically similar; it must meet the support criteria. The verifier (an LLM or rule-based evaluator) is itself evaluated against human labels and is not assumed to be correct.

6.4. Coverage-directed retrieval

The system first retrieves a bounded passage set, then uses component states to decide what to do next:

- Missing requirements trigger targeted retrieval.
- Partial requirements trigger searches for additional or more decisive evidence.
- Supported requirements are retained and not re-searched without reason.
- Unresolved requirements are carried forward until supported, the budget ends, or the stopping policy judges further search unlikely to help.

6.5. Stopping policy

Retrieval stops when:

1. All required components meet the support criteria;
2. The maximum iterations are reached;
3. The token, inference-call, or latency budget is exhausted; or
4. Repeated retrieval yields no meaningful new evidence under a predefined diminishing-returns rule.

If requirements remain unsupported, the answer is reported as partial with the unresolved requirements listed. An answer is never labelled complete merely because a verifier is confident about the context overall. Iteration limits and token budgets will be set from the pilot and frozen before test evaluation.

6.6. Coverage and answer-item measures

Let R_gold(q) be the gold requirement set and S(q) the subset whose support is established **by an evaluator independent of the system** (human annotation, or an independent judge validated against humans; never the system's internal verifier).

`Coverage(q) = |S(q) ∩ R_gold(q)| / |R_gold(q)|`

For answer items, with P the predicted and G the gold item set:

`Precision = |P ∩ G| / |P|`, `Recall = |P ∩ G| / |G|`, `F1 = 2 * Precision * Recall / (Precision + Recall)`

Coverage requires gold requirements and evidence-support annotations, so it is computed only where they exist. It is always reported with requirement-extraction precision and recall, verification accuracy, false-support rate, and unsupported-claim rate,

so that declaring everything supported cannot yield a high score. Empty prediction sets and no-valid-answer questions follow a predefined scoring policy.

7. Experimental Design

7.1. Datasets (core and optional)

Core.

- **MuSiQue:** compositional multi-hop questions (the single multi-hop benchmark in the core set; HotpotQA or 2WikiMultiHopQA may be added if time allows).
- **QAMPARI:** open-domain list questions with answers spread over many passages; the main benchmark for answer-item recall.
- **GUARD-RAG Completeness Set:** custom, corpus-bounded set (below).

Optional extension. QUEST (multi-constraint entity-seeking queries), added only after the core experiments are complete.

Multi-hop datasets are not exhaustive-list benchmarks, since their answers are typically short. Public gold answers can also be incomplete (QAMPARI lists are Wikipedia-derived), so apparent false positives will be sampled and manually reviewed, and adjusted precision reported alongside raw precision.

Metric availability by dataset. Item precision, recall, and F1 apply to QAMPARI and the Completeness Set; Exact Match and F1 to MuSiQue. **Gold-requirement coverage and premature-stopping rate are measured only on the Completeness Set**, because deciding whether an omission was "recoverable" needs the annotation protocol; public datasets do not provide it.

7.2. GUARD-RAG Completeness Set

- **Size:** initial target of 200–300 questions, to be revised after the pilot.
- **Corpus:** a bounded collection of publicly available, openly licensed text documents (for example, government or institutional reports and policy documents), fixed as a snapshot. The exact domain is chosen in Weeks 3–4 so that completeness is operationally defined.
- **Types (about four):** exhaustive or multi-answer lists; multi-part comparisons and questions; multi-document evidence synthesis; insufficient-evidence questions whose correct outcome is a partial answer or abstention. Initial retrieval difficulty is a tag within these types, not a separate type.

- **Per-type minimum:** with 200–300 questions each type has roughly 50–75 questions. After the pilot we fix a minimum per type sufficient for the planned confidence intervals, or merge types if resources fall short.
- **Annotators:** at least two trained annotators label independently a subset; a third adjudicates disagreements. The pilot estimates annotation time per question so the final size is feasible.

Questions may be drafted manually or with LLM assistance, but every final question and gold label is human-verified. Annotation records the question and corpus scope, required components, acceptable answer variants, supporting passages, each component's status under the available evidence, and the criteria for complete, partial, or unsupported answers. Inter-annotator agreement is reported. Development and test splits are separated, and test annotations are never used for tuning.

7.3. Models and retrieval setup

- **Generator:** one open-weight instruction-tuned model in the 7–13B range and one API-served model, to test whether results depend on a single model. Exact versions and prompts are recorded.
- **Retriever:** BM25 plus a dense embedding retriever, with optional reranking, shared across all systems.
- **Verifier and judge separation:** the model performing support verification inside GUARD-RAG differs from the model used as the final evaluation judge, preferably from a different model family and with different prompts. Judge agreement with humans is reported on a representative subset.

7.4. Baselines

#	System	Purpose
1	Fixed top-k vector RAG	Standard reference
2	Hybrid lexical + semantic RAG	Stronger retrieval reference
3	Adaptive-RAG (if a faithful implementation is feasible)	Complexity-based adaptive method
4	CRAG or comparable corrective baseline	Retrieval-quality control
5	IRCoT (supplementary where feasible)	Iterative retrieval plus decomposition

#	System	Purpose
6	Decomposition-only (no persistent state)	Does decomposition alone explain gains?
7	Sufficiency-only (context-level adequacy decision, no per-component tracking)	Does context-level sufficiency explain gains?
8	**Compute-matched baseline** (same retrieval calls and tokens as GUARD-RAG, spent by larger k or diverse extra retrieval, with no coverage state)	Are gains just from more retrieval?
9	Full GUARD-RAG	Proposed system

Baselines 6–8 are built from GUARD-RAG's own modules with the same retriever, generator, corpus, verifier (where applicable), and budget, so they are the most direct tests of H3 and H5. Self-RAG, FLARE, GraphRAG, and Hyper-RAG are supplementary and included only if they can be reproduced fairly. Systems share questions, corpus snapshot, and predeclared budgets; any other differences are documented.

7.5. Ablations

- **Full model:** requirement extraction, persistent state, verification, targeted retrieval, stopping policy.
- **Without persistent coverage:** keep decomposition and retrieval, drop persistent state.
- **Without sufficiency verification:** track requirements without the support check.
- **Without targeted retrieval:** keep the state but use the baseline retrieval policy.
- **Without early stopping:** run to the maximum budget.
- **Oracle requirements:** replace predicted requirements with gold requirements. This estimates the loss from extraction errors and is an analytical upper bound, not a deployable system.

7.6. Fairness and reproducibility

All primary comparisons use the same corpus snapshot, generator, retrieval settings, and predeclared budgets. Hyperparameters are tuned on development data only. We report model versions, prompts, retrieval configuration, seeds where relevant, and compute cost.

8. Evaluation Metrics

Answer quality: Exact Match and token-level F1 where appropriate; item precision, recall, and F1 for multi-answer tasks (primary: item F1); human evaluation of correctness and completeness on a stratified sample.

Evidence quality: gold-requirement coverage (Completeness Set); claim-support verification accuracy; unsupported-claim rate; citation correctness; false-support rate.

Retrieval-control quality (Completeness Set): premature-stopping rate, defined as stopping while at least one requirement is recoverable from the permitted corpus within the budget, according to the annotation protocol; requirement-extraction precision, recall, and F1; unnecessary retrieval rate; number of unresolved requirements; accuracy of complete-vs-partial reporting.

Efficiency: retrieval calls and iterations; input and output tokens; model calls and estimated cost; end-to-end latency; indexing cost where methods differ. Results are shown as quality–cost curves where feasible.

9. Statistical Analysis

The primary analysis compares full GUARD-RAG against the decomposition-only, sufficiency-only, and compute-matched baselines on item F1. We report confidence intervals and paired significance tests suited to the metric, a pre-specified correction for multiple primary comparisons, and effect sizes alongside p-values. A query-type analysis examines list, multi-hop, multi-part, and insufficient-evidence questions, and we report whether recall gains cost precision, latency, or inference cost. Repeated runs are used where model variability requires them.

10. Pilot Study and Go/No-Go Criteria

In Weeks 1–2, in parallel with the literature review, we run fixed top-k RAG on 20–30 multi-component questions and record how often required components are missing from the final answer or evidence, how long annotation takes, and initial inter-annotator agreement.

- **Go:** if missing or unsupported components are common enough to leave clear headroom (a threshold fixed in advance, for example in a substantial minority of questions), proceed as planned and use the pilot as the Introduction's motivating evidence.

- **Revise:** if the problem is rare overall but concentrated in particular question types, narrow the scope to those types and revise H1 and H4.
- **No-go:** if the problem is rare everywhere, or the novelty review shows a substantially equivalent published method, reconsider the research question before implementation.

11. Scope and Limitations

The first study covers text-based retrieval and multi-document question answering. It does not address visual flowchart understanding, multimodal retrieval, automatic resolution of conflicting clinical guidelines, or graph and hypergraph knowledge construction.

Completeness is defined relative to the specified corpus and query requirements; the method cannot find facts absent from the corpus. Requirement extraction and evidence verification can introduce errors, public benchmark labels may omit valid answers, and the custom set may not represent every real information need. We address these through explicit corpus boundaries, human-verified annotation, oracle experiments, error analysis, and transparent reporting of unresolved requirements.

12. Expected Contributions

1. **A persistent component-level evidence state** recording whether each required component is missing, partial, supported, or unresolved.
2. **A coverage-directed retrieval controller** that prioritises retrieval and decides when to stop or return a partial answer.
3. **A controlled evaluation** isolating the value of persistent coverage tracking from decomposition, context-level sufficiency, and extra retrieval.
4. **A reproducible evaluation resource:** a corpus-bounded annotated set and protocol for answer-item completeness, evidence support, premature stopping, and cost, subject to annotation and release constraints.

13. Work Plan (16 weeks)

Weeks	Activity
1–2	Pilot on 20–30 questions (go/no-go); systematic literature search and novelty matrix; benchmark and model selection
3–4	Corpus selection; annotation guidelines; Completeness Set pilot; fix per-type counts, budgets, and effect-size thresholds
5–6	Annotation of the Completeness Set; evaluation scripts; baseline setup
7–9	Implement requirement extraction, persistent state, verification, and bounded retrieval control
10–12	Matched baselines (including decomposition-only, sufficiency-only, compute-matched), ablations, oracle experiment
13–14	Final evaluation, statistics, efficiency profiling, error analysis, human evaluation
15–16	Paper writing, citation verification, reproducibility materials

If annotation or baseline reproduction runs long, the study keeps the controlled core comparison and trims supplementary experiments (QUEST, Self-RAG, FLARE, graph baselines) rather than weakening evaluation quality.

14. Risk Mitigation

Risk	Mitigation
Insufficient novelty	Systematic review before fixing claims; revise the hypothesis if an equivalent method exists
Problem rare in practice	Week 1–2 pilot with pre-set go/no-go thresholds

Risk	Mitigation
Requirement-extraction errors	Report extraction precision and recall; run the oracle experiment
Gains explained by extra retrieval	Compute-matched baseline at equal calls and tokens
Circular evaluation	Independent evaluator for S(q); different model for verifier and judge
Incomplete benchmark labels	Manual review of sampled false positives; adjusted precision; custom set
Excessive cost	Explicit iteration, token, and latency budgets
Recall–precision imbalance	Primary metric is item F1; report precision and unsupported claims
Baselines hard to reproduce	Build matched baselines in-house; document differences
Annotation workload	Use the pilot to size the set; merge types if needed

15. Conclusion

GUARD-RAG investigates whether persistent, per-component evidence tracking is an effective retrieval-control signal for answer completeness under fixed resources. The scientific test is not whether iterative retrieval can help in general, but whether explicit coverage state adds value beyond decomposition, context-level sufficiency, and simply retrieving more. The study succeeds if it produces a rigorous, reproducible answer, even if the mechanism does not consistently beat simpler alternatives.

References (preliminary; verify before submission)

1. Jeong, S. et al. *Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity.* NAACL 2024. https://aclanthology.org/2024.naacl-long.389/
2. Asai, A. et al. *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.* ICLR 2024. https://openreview.net/forum?id=hSyW5go0v8

3. Yan, S.-Q. et al. *Corrective Retrieval Augmented Generation.* 2024. https://arxiv.org/abs/2401.15884
4. Jiang, Z. et al. *Active Retrieval Augmented Generation* (FLARE). EMNLP 2023. https://aclanthology.org/2023.emnlp-main.495/
5. Trivedi, H. et al. *Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions* (IRCoT). ACL 2023. https://aclanthology.org/2023.acl-long.557/
6. Joren, H. et al. *Sufficient Context: A New Lens on Retrieval Augmented Generation Systems.* ICLR 2025. https://arxiv.org/abs/2411.06037
7. Amouyal, S. et al. *QAMPARI: A Benchmark for Open-domain Questions with Many Answers.* GEM 2023. https://aclanthology.org/2023.gem-1.9/
8. Malaviya, C. et al. *QUEST: A Retrieval Dataset of Entity-Seeking Queries with Implicit Set Operations.* ACL 2023.
9. Trivedi, H. et al. *MuSiQue: Multihop Questions via Single-hop Question Composition.* TACL 2022. https://aclanthology.org/2022.tacl-1.31/
10. Yang, Z. et al. *HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering.* EMNLP 2018. https://aclanthology.org/D18-1259/
11. Ho, X. et al. *Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps* (2WikiMultiHopQA). COLING 2020.
12. Press, O. et al. *Measuring and Narrowing the Compositionality Gap in Language Models* (Self-Ask). Findings of EMNLP 2023.
13. Edge, D. et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization.* 2024.

This list is a starting point. The final manuscript should include a verified, systematic comparison of the closest related methods and the exact implementation versions used.